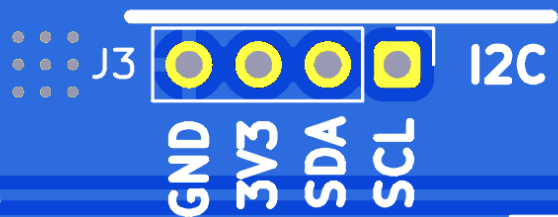
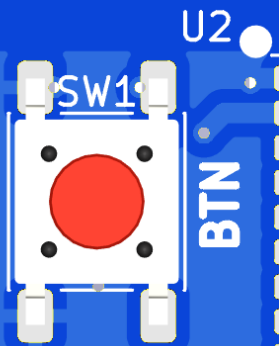
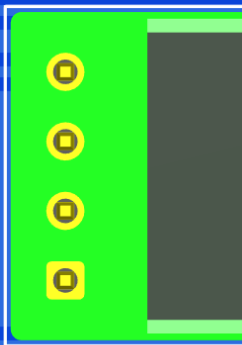
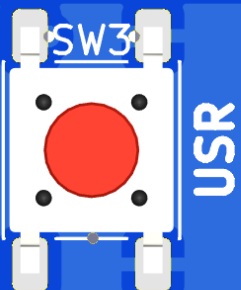


ERASMUS+
GEMS
SAPPHIRE



U2



ROS (Robot Operating System) Fundamentals

Georgopoulos Dimitris
Teaching Factory Competence Center



Learning Objectives

1

Understand what ROS is and why it's used

3

Describe the evolution from ROS 1 to ROS 2

2

Identify core ROS concepts

4

Form a basic mental model of how robots use ROS

Intro to ROS

- What ROS is?
- Why it matters?
- ROS 1 → ROS 2 evolution
- Key concepts you must know

What Is ROS?

ROS = Robot Operating System

But not an actual operating system! It's a:

- Middleware for robotics
- Communication framework for robot components
- Set of tools & libraries for building robot applications

Goal: Make robotics development easier, reusable, and modular.

Why Use ROS?

Encourages modular robot design

Large ecosystem of drivers, packages, algorithms

Works on real robots & simulation

Big community + industry adoption

Makes building robots faster and cheaper

ROS History and Evolution

ROS 1 (2010–2020+)

- First generation
- Widely adopted
- Not designed for real-time or multi-robot systems
- Examples: ROS Kinetic, ROS Melodic, ROS Noetic (final LTS)

Transition to ROS 2

Created to address ROS 1 limitations:

- Real-time capability
- Security
- Multi-robot support
- Better communications (DDS)

ROS 2 Overview

ROS 2 releases (all named after animals)

- Foxy Fitzroy (2020) – First long-term stable
- Galactic (2021)
- Humble Hawksbill (2022, LTS)
- Iron Irwini (2023)
- Jazzy Jalisco (2024, LTS) – Latest long-term version

ROS 2 is now the mainline and future of ROS.

Humble Hawksbill (2022–2027)

Humble Hawksbill (2022–2027)

- Stable
- Ubuntu 22.04
- Widely used in production

Jazzy Jalisco (2024–2029)

- Latest LTS
- Ubuntu 24.04
- Strong community momentum

Core ROS Concepts (Beginner Friendly)

Nodes

- Small programs that do one thing
- Example: camera node, motor control node

Topics

- Channels that nodes use to send data
- Example: /camera/image_raw

Publishers & Subscribers

- Publisher → sends data
- Subscriber → receives data

Messages

- The data format sent through topics
- Example: sensor_msgs/Image

More ROS Fundamentals

Services

- Request/response actions
- Good for non-continuous tasks
- Example: “Turn LED on”

Actions

- Long-running tasks
- Example: “Move arm to position”

Parameters

- Configurations for nodes
- Example: robot speed = 0.5

Packages

- ROS software units that contain nodes, configs, etc.

ROS Tools (Worth Showing)

rqt

- Graphical visualization & debugging tools

rviz2

- 3D visualization (sensors, robots, maps)

Gazebo / Ignition

- Physics simulation for robots

colcon

- Build system for ROS 2

ROS Example (Simple Mental Model)

Robot = Many Nodes Talking to Each Other

- Camera Node → publishes images
- Object Detection Node → subscribes to images
- Motion Planner → receives results
- Motor Driver Node → executes movement

Everything communicates through topics → modular and flexible.

What Can You Build with ROS?

- Autonomous mobile robots
- Manipulator arms
- Drones
- Self-driving cars (research)
- Service robots & AGVs
- Humanoids (NVIDIA GR00T, Unitree, etc.)

ROS Ecosystem & Community

- Global open-source community
- Thousands of plugins & packages
- Supported by universities, hobbyists, and major companies
- ROSCon conference every year

ROS 1 vs ROS 2 (Quick Summary)

Feature	ROS 1	ROS 2
Underlying transport	Custom	DDS (industry standard)
Real-time	Limited	Yes
Security	No	Yes
Multi-robot	Hard	Built-in
Long-term future	Maintenance mode	Active development

Which Version Should Beginners Use?

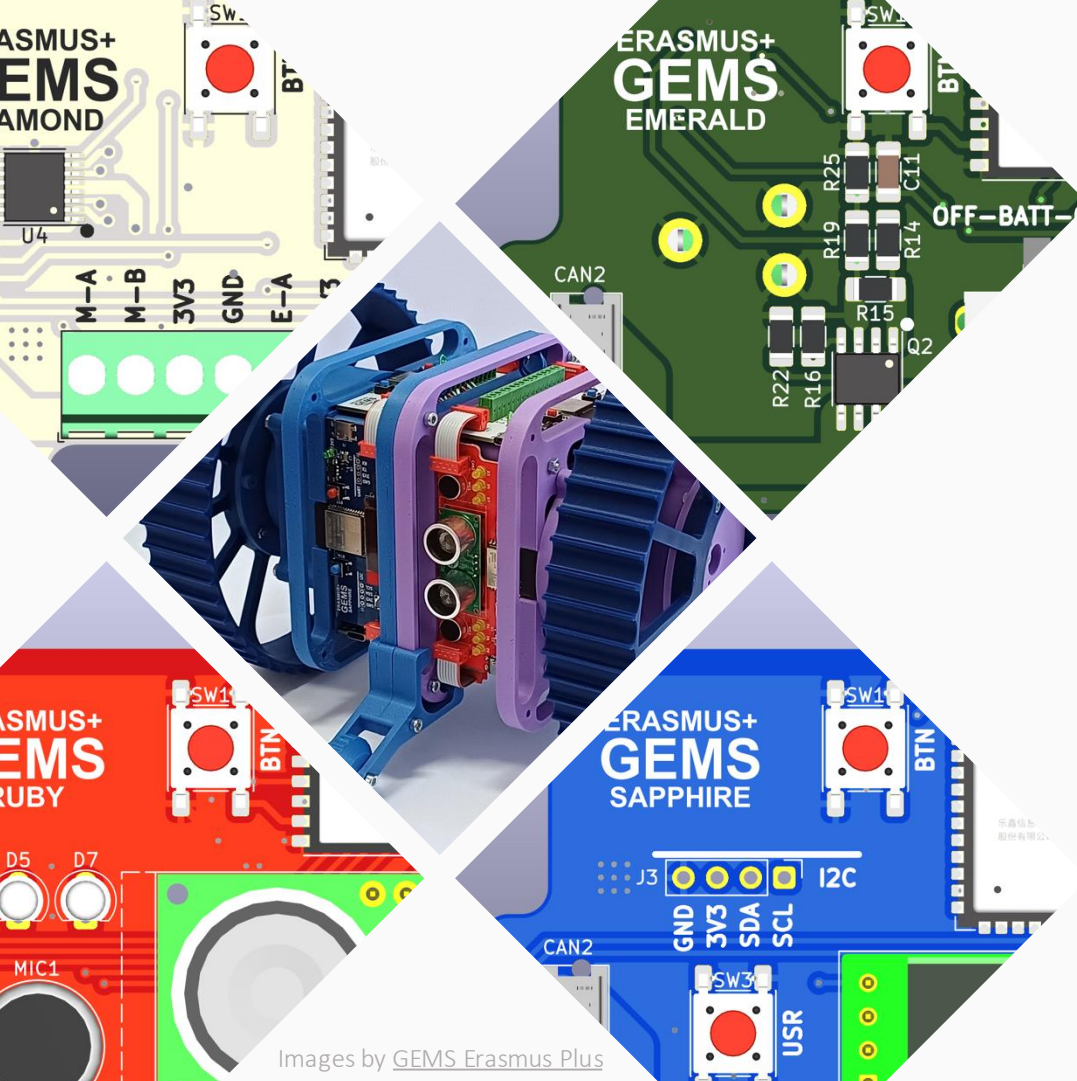
Recommended: ROS 2 Jazzy (LTS)

- Latest stable
- Best long-term support
- Tutorials and community trending toward Jazzy

Alternative: ROS 2 Humble for systems running Ubuntu 22.04

Where to Learn More

- **docs.ros.org**
- **ROS 2 Tutorials**
- **The Construct** (simulation-based learning)
- **GitHub examples**
- **ROS Answers** (Q&A forum)



Images by [GEMS Erasmus Plus](#)

Thank you for watching!

This video was created by Teaching Factory Competence Center for the GEMS Erasmus+ project (a collaboration between University of Ljubljana, University of Alcala, Teaching Factory Competence Center, and Delft University of Technology). It is released under a **Creative Commons Attribution Non Commercial Share Alike 4.0 International License**



**Co-funded by
the European Union**

Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or CMEPIUS. Neither the European Union nor the granting authority can be held responsible for them.